

dbms - Question Bank Answers

Q1 (10 marks): Explain the concept of normalization. Discuss the different types of normal forms from 1NF to BCNF with suitable examples.

Normalization ===== Normalization is a database design technique used to reduce data redundancy and improve data integrity by organizing data into tables with minimal data repetition. Types of Normal Forms -----
1NF (First Normal Form) * Eliminates repeating groups * Each row has a unique set of values * Example: A table with a column for employee names and a column for employee addresses is not in 1NF because an employee can have multiple addresses.
2NF (Second Normal Form) * Eliminates partial dependencies * Each non-key attribute depends on the entire primary key * Example: A table with a primary key of customer ID and columns for customer name, order ID, and order date is not in 2NF because order ID is not dependent on the customer ID.
3NF (Third Normal Form) * Eliminates transitive dependencies * If a table is in 2NF, and a non-key attribute depends on another non-key attribute, then it should be moved to a separate table * Example: A table with a primary key of customer ID and columns for customer name, order ID, and order date is not in 3NF because order date depends on order ID, which depends on customer ID.
BCNF (Boyce-Codd Normal Form) * A table is in BCNF if and only if it is in 3NF and there are no transitive dependencies * Example: A table with a primary key of customer ID and columns for customer name, order ID, and order date is in BCNF because there are no transitive dependencies. Conclusion =====
Normalization is a crucial step in database design that helps reduce data redundancy and improve data integrity. By following the different types of normal forms, we can ensure that our database is well-organized and efficient.

Q2 (10 marks): Consider the relation: *PRODUCT (P_id, P_name, Category, Price, Quantity)* Write SQL queries for the following: Count the total number of products in the *PRODUCT* table. Display the names of products whose price is greater than 1000. Find the highest and lowest product prices. Display all products in ascending order of price. Display only product names and prices. Calculate the total quantity of all products available in stock.

Introduction ----- Transaction processing systems are designed to handle large databases and hundreds of concurrent users executing database transactions. These systems require high availability and fast response time for hundreds of concurrent users. Transaction Properties -----
A transaction is a logical unit of database processing that includes database commands such as retrievals, insertions, deletions, and updates. The properties of a transaction include: * Atomicity: A transaction is treated as a single, indivisible unit of work. * Consistency: A transaction must maintain the consistency of the database. * Isolation: A transaction must be executed independently of other transactions. * Durability: Once a transaction is committed, its effects must be permanent. Creation of Schedule and Maintaining Schedule Equivalence -----
A schedule is a sequence of transactions executed by a database system. Schedule equivalence is the property of two schedules being equivalent in terms of their effects on the database. Protocols Used for Locking Objects -----
Two common protocols used for locking objects are: SQL Queries for *PRODUCT* Table -----
``sql -- Count the total number of products in the *PRODUCT* table
SELECT COUNT(*) FROM *PRODUCT*; -- Display the names of products whose price is greater than 1000
SELECT P_name FROM *PRODUCT* WHERE Price > 1000; -- Find the highest and lowest product prices
SELECT MAX(Price), MIN(Price) FROM *PRODUCT*; -- Display all products in ascending order of price
SELECT * FROM *PRODUCT* ORDER BY Price ASC; -- Display only product names and prices
SELECT P_name, Price FROM *PRODUCT*; -- Calculate the total

quantity of all products available in stock SELECT SUM(Quantity) FROM PRODUCT; ``

Conclusion ----- Transaction processing systems are designed to handle large databases and hundreds of concurrent users executing database transactions. The properties of a transaction include atomicity, consistency, isolation, and durability. Two common protocols used for locking objects are 2PL and Strict 2PL. The SQL queries provided above demonstrate various operations that can be performed on the PRODUCT table.

Q3 (10 marks): Given the relation: LIBRARY (BookID, BookTitle, AuthorID, AuthorName, PublisherID, PublisherName) Functional Dependencies: BookID → BookTitle, PublisherID AuthorID → AuthorName PublisherID → PublisherName a) Identify all functional dependencies. b) Determine the highest normal form achieved. c) Normalize the relation step-by-step from 1NF to 3NF.

Functional Dependencies The given relation LIBRARY has the following functional dependencies: - BookID → BookTitle - PublisherID → PublisherName - AuthorID → AuthorName - PublisherID → PublisherName Determination of Normal Form To determine the highest normal form achieved, we need to analyze the relation for each normal form. Normalization Steps To normalize the relation step-by-step from 1NF to 3NF: 1. 1NF : The relation is already in 1NF as there are no repeating groups. 2. 2NF : We need to identify the non-prime attributes that depend on the primary key. In this case, BookTitle depends on BookID, AuthorName depends on AuthorID, and PublisherName depends on PublisherID. However, PublisherID is a non-prime attribute that depends on the primary key. To achieve 2NF, we need to split the relation into two relations: - BOOK(BookID, BookTitle) - PUBLISHER(PublisherID, PublisherName) - AUTHOR(AuthorID, AuthorName) 3. 3NF : We need to identify the non-prime attributes that depend on another non-prime attribute. In this case, AuthorName depends on AuthorID, which is a non-prime attribute. To achieve 3NF, we need to split the relation into two relations: - AUTHOR(AuthorID, AuthorName) - BOOK(BookID, BookTitle, AuthorID) - PUBLISHER(PublisherID, PublisherName) The highest normal form achieved is 3NF.

Q4 (10 marks): Differentiate between assertions and triggers in SQL. Discuss their applications in maintaining database integrity.

Introduction ----- Assertions and triggers are two mechanisms used in SQL to maintain database integrity. They help ensure that the data in a database remains consistent and accurate. Assertions ----- Triggers ----- Applications ----- * Assertions are used to enforce constraints on the data, such as checking the age of a person. * Triggers are used to enforce constraints on the data and perform actions based on specific events, such as updating the salary of an employee when their age increases. * Assertions are typically used to enforce simple constraints, while triggers are used to enforce more complex constraints and perform actions. Conclusion ----- Assertions and triggers are two important mechanisms used in SQL to maintain database integrity. While assertions are used to enforce simple constraints, triggers are used to enforce more complex constraints and perform actions based on specific events. By using these mechanisms, database administrators can ensure that the data in a database remains consistent and accurate.

Q5 (10 marks): Define normal form? Explain 2NF and 3NF with an example

Normal Form ===== Normal form is a concept in database design that ensures data is organized in a way that minimizes data redundancy and dependency. It is a set of rules that define

how to structure a relational database to prevent data anomalies. First Normal Form (1NF) ----- * A table is in 1NF if it contains only atomic values, i.e., each cell contains a single value. * No repeating groups or arrays are allowed. * Each row must have a unique combination of values. Second Normal Form (2NF) ----- A table is in 2NF if it meets the following conditions: * It is in 1NF. * No non-key attribute depends on a part of the primary key. Third Normal Form (3NF) ----- A table is in 3NF if it meets the following conditions: * It is in 2NF. * No non-key attribute depends on another non-key attribute. Example ----- Suppose we have a table called "Orders" with the following columns: Order ID, Customer ID, Order Date, Product ID, and Quantity. This table is not in 3NF because the Product ID depends on the Order ID, which is a non-key attribute. To normalize this table, we can create two separate tables: "Customers" and "Orders". This normalized table is in 3NF because there are no transitive dependencies.

Q6 (10 marks): Consider relation R (a, b, c, d, e) functional dependencies. F = {A → B, BC, D → E}. Find the candidate key?

Candidate Key ===== A candidate key is a set of attributes in a relation that uniquely identifies each tuple in the relation. Properties of Candidate Key ----- * A candidate key must be a superkey. * A candidate key must have no redundant attributes. * A candidate key must be minimal, i.e., no proper subset of the candidate key can be a superkey. Finding Candidate Key ----- To find the candidate key, we need to find the minimal superkey. Step 1: Find all possible superkeys ----- A superkey is a set of attributes that uniquely identifies each tuple in the relation. Step 2: Find the minimal superkey ----- A candidate key must be a minimal superkey. Candidate Keys ----- The candidate keys are the minimal superkeys. Conclusion ----- The candidate key for the relation R (a, b, c, d, e) with functional dependencies F = {A → B, BC, D → E} is {A, BC}.

Q7 (10 marks): Consider the relation: SALES (Sale_id, Product_Name, Quantity, Price, Salesperson) Write SQL queries for the following: Find the total number of sales records. Calculate the total quantity sold. Find the highest product price. Calculate the average product price. Display only Product_Name and Price. Display only Salesperson and Quantity.

Introduction ----- Transaction processing systems are designed to handle large databases and hundreds of concurrent users executing database transactions. These systems require high availability and fast response time for hundreds of concurrent users. Objectives ----- * To study transaction properties * To study creation of schedule and maintaining schedule equivalence * To check whether the given schedule is serializable or not * To study protocols used for locking objects Single-User versus Multiuser Systems ----- Multiprogramming ----- Multiprogramming can be achieved by: * Interleaved execution * Parallel Processing SQL Queries for SALES Relation ----- #### Find the total number of sales records ```sql SELECT COUNT(*) FROM SALES; ``` #### Calculate the total quantity sold ```sql SELECT SUM(Quantity) FROM SALES; ``` #### Find the highest product price ```sql SELECT MAX(Price) FROM SALES; ``` #### Calculate the average product price ```sql SELECT AVG(Price) FROM SALES; ``` #### Display only Product_Name and Price ```sql SELECT Product_Name, Price FROM SALES; ``` #### Display only Salesperson and Quantity ```sql SELECT Salesperson, Quantity FROM SALES; ``` Conclusion ----- Transaction processing systems are designed to handle large databases and hundreds of concurrent users executing database transactions. SQL queries can be used to perform various operations on the SALES relation, such as finding the total number of sales records, calculating the total quantity sold, and displaying specific columns.

Q8 (10 marks): A relation R (A, B, C, D, E) has the following functional dependencies: $A \rightarrow BC$ $C \rightarrow D$ $D \rightarrow E$ 1. Find A^+ (closure of A). 2. Determine the candidate key(s). 3. Explain whether transitive dependencies exist.

1. Find A^+ (closure of A) The closure of A, denoted as A^+ , is the set of all attributes that can be determined from A using the given functional dependencies. Given functional dependencies: - $A \rightarrow BC$ - $C \rightarrow D$ - $D \rightarrow E$ To find A^+ , we start with A and apply the given functional dependencies: 1. $A \rightarrow BC$ (directly given) 2. Since C is in A^+ , we can find D using $C \rightarrow D$: $A^+ \rightarrow D$ 3. Since D is in A^+ , we can find E using $D \rightarrow E$: $A^+ \rightarrow E$ Therefore, $A^+ = A + BC + D + E = A + BCDE$ 2. Determine the candidate key(s) A candidate key is a set of attributes that uniquely identifies each tuple in the relation. Given functional dependencies: - $A \rightarrow BC$ - $C \rightarrow D$ - $D \rightarrow E$ We can see that A is a candidate key because it uniquely determines all other attributes (BCDE). 3. Explain whether transitive dependencies exist A transitive dependency exists if an attribute Y is functionally dependent on an attribute X, and X is functionally dependent on an attribute Z. Given functional dependencies: - $A \rightarrow BC$ - $C \rightarrow D$ - $D \rightarrow E$ We can see that there are transitive dependencies: - $A \rightarrow BC$ (directly given) - $C \rightarrow D$ (directly given) - $D \rightarrow E$ (directly given) Therefore, transitive dependencies exist.

Q9 (10 marks): What is normalization? Explain the different types of normalization. Explain from First Normal Form (1NF) to Third Normal Form (3NF) with suitable examples.

Normalization ===== Normalization is the process of organizing data in a database to minimize data redundancy and dependency. It involves dividing large tables into smaller tables and linking them through relationships. Types of Normalization ----- First Normal Form (1NF) ----- * Each table cell must contain a single value. * Example: A table with multiple values in a single cell is not in 1NF. Normalized Table (1NF) Second Normal Form (2NF) ----- * Each non-key attribute in a table must depend on the entire primary key. * Example: A table with a non-key attribute that depends on only a part of the primary key is not in 2NF. Normalized Table (2NF) Third Normal Form (3NF) ----- * If a table is in 2NF, and a non-key attribute depends on another non-key attribute, then it should be moved to a separate table. * Example: A table with a non-key attribute that depends on another non-key attribute is not in 3NF. Normalized Table (3NF)

Q10 (10 marks): Consider the relation: EMPLOYEE(E_id, E_name, Age, Salary) Write an SQL query to count the total number of employees in the EMPLOYEE table. Write an SQL query to count the number of employee names available in the EMPLOYEE table. Write an SQL query to find the maximum and minimum age of employees in the EMPLOYEE table. Write an SQL query to display the salaries of all employees in ascending order. Write an SQL query to display only employee names and salaries.

Transaction Processing ===== 5.1 Objectives ----- * To study transaction properties * To study creation of schedule and maintaining schedule equivalence * To check whether the given schedule is serializable or not * To study protocols used for locking objects 5.2 Introduction to Transaction Processing ----- ### 5.2.1 Single-User versus Multiuser Systems Transaction Properties ----- * Atomicity: A transaction is treated as a single unit of work * Consistency: The database is left in a consistent state after a transaction * Isolation: Multiple transactions can execute concurrently without interfering with each other * Durability: Once a transaction is committed, its effects are permanent SQL Queries ----- ### 1. Count the total number of employees ``sql SELECT COUNT(*) FROM

EMPLOYEE; ````### 2. Count the number of employee names ````sql SELECT COUNT(E_name)
FROM EMPLOYEE; ````### 3. Find the maximum and minimum age of employees ````sql SELECT
MAX(Age), MIN(Age) FROM EMPLOYEE; ````### 4. Display salaries in ascending order ````sql
SELECT Salary FROM EMPLOYEE ORDER BY Salary ASC; ````### 5. Display only employee
names and salaries ````sql SELECT E_name, Salary FROM EMPLOYEE; ````

Q11 (10 marks): Consider the relation: $R(A, B, C, D, E)$ with the following set of functional dependencies: $A \rightarrow BC$ $B \rightarrow D$ $CD \rightarrow E$ Compute the closure of the following attribute sets and candidate key : 1. $(A)^+$ 2. $(CD)^+$

Introduction ----- Transaction processing systems are designed to handle large databases and hundreds of concurrent users executing database transactions. These systems require high availability and fast response time. Single-User versus Multiuser Systems
----- Multiprogramming ----- Multiprogramming can be achieved by: * Interleaved execution * Parallel Processing Closure of Attribute Sets -----
Given the relation $R(A, B, C, D, E)$ with the following set of functional dependencies: $A \rightarrow BC$ $B \rightarrow D$ $CD \rightarrow E$ 1. $(A)^+$ ----- To compute the closure of (A) , we need to find all the attributes that are functionally dependent on A . * $A \rightarrow BC$ (given) * Since $B \rightarrow D$ (given), we can infer that $A \rightarrow D$ * Since $CD \rightarrow E$ (given), we can infer that $A \rightarrow E$ (because C is functionally dependent on A) Therefore, the closure of (A) is (A, B, C, D, E) . 2. $(CD)^+$ ----- To compute the closure of (CD) , we need to find all the attributes that are functionally dependent on CD . * $CD \rightarrow E$ (given) * Since $A \rightarrow BC$ (given), we can infer that $CD \rightarrow BC$ (because C is part of CD) * Since $B \rightarrow D$ (given), we can infer that $CD \rightarrow D$ Therefore, the closure of (CD) is (CD, E, B, C, D) . Candidate Key ----- A candidate key is a set of attributes that uniquely identifies each tuple in the relation. * Since (A, B, C) uniquely identifies each tuple in the relation, it is a candidate key. * Since (CD) uniquely identifies each tuple in the relation, it is also a candidate key. Therefore, the candidate key is (A, B, C) and (CD) .

Q12 (10 marks): Demonstrate working of Assertion Triggers in database? Explain with an example.

Introduction ----- Assertion and Triggers are two powerful features in database management systems that help maintain data consistency and integrity. Assertion ----- An assertion is a statement that checks the validity of data in a database. It is a constraint that is applied to a database to ensure that the data remains consistent. Types of Assertions -----
Example of Assertion ----- ````sql CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, Name VARCHAR(255), Salary DECIMAL(10, 2)); CREATE ASSERTION Check_Salary CHECK (Salary > 0); ```` In this example, the assertion `Check\_Salary` ensures that the salary of each employee is greater than 0. Triggers ----- A trigger is a set of actions that are executed automatically when a specific event occurs in a database. Types of Triggers -----
Example of Trigger ----- ````sql CREATE TABLE Orders (OrderID INT PRIMARY KEY, OrderDate DATE, Total DECIMAL(10, 2)); CREATE TRIGGER Check_Total BEFORE INSERT ON Orders FOR EACH ROW BEGIN IF NEW.Total SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid total'; END IF; END; ```` In this example, the trigger `Check\_Total` checks the total of each order before it is inserted into the database. If the total is less than 0, it raises an error.

Q13 (10 marks): Consider the relation: $STUDENT_COURSE(Student_ID, Student_Name, Course_ID, Course_Name, Instructor_Name, Marks)$ Assume the following functional dependencies: $Student_ID \rightarrow Student_Name$

Course_ID → Course_Name, Instructor_Name (Student_ID, Course_ID) → Marks Identify the candidate key. Determine whether the relation is in 1NF and 2NF. Normalize the relation up to 2NF.

Candidate Key Identification The candidate key is the set of attributes that uniquely identify each tuple in the relation. In the given relation STUDENT_COURSE, the candidate key is Student_ID and Course_ID . **Functional Dependencies** The functional dependencies are: - Student_ID → Student_Name - Course_ID → Course_Name, Instructor_Name - (Student_ID, Course_ID) → Marks **1NF and 2NF** A relation is in 1NF if each attribute in the relation is a single-valued attribute. A relation is in 2NF if it is in 1NF and each non-prime attribute in the relation depends on the entire primary key. The relation STUDENT_COURSE is not in 1NF because the attribute Marks depends on both Student_ID and Course_ID , which are not single-valued attributes. **Normalization up to 2NF** To normalize the relation up to 2NF, we need to split the relation into two relations: 1. STUDENT (Student_ID, Student_Name) 2. COURSE (Course_ID, Course_Name, Instructor_Name, Marks) The STUDENT relation has Student_ID as the primary key. The COURSE relation has Course_ID as the primary key and Marks as a non-prime attribute. **Conclusion** The candidate key of the relation STUDENT_COURSE is Student_ID and Course_ID . The relation is not in 1NF because the attribute Marks depends on both Student_ID and Course_ID . The relation is normalized up to 2NF by splitting it into two relations: STUDENT and COURSE.

Q14 (10 marks): Explain the concept of transaction processing in DBMS. Discuss the need and importance of transactions in database systems.

Could not generate answer: Error code: 429 - {'error': {'message': 'Rate limit reached for model `llama-3.1-8b-instant` in organization `org\_01kzh2evnb6rmysy0q0ea70s5` service tier `on\_demand` on tokens per minute (TPM): Limit 6000, Used 4697, Requested 1547. Please try again in 2.44s. Need more tokens? Upgrade to Dev Tier today at <https://console.groq.com/settings/billing>', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}

Q15 (10 marks): Illustrate the working of the Two-Phase Locking (2PL) protocol for concurrency control using a suitable example.

Introduction ----- The Two-Phase Locking (2PL) protocol is a concurrency control mechanism used in database systems to manage concurrent access to shared resources. It ensures that transactions execute in a way that prevents conflicts and maintains the consistency of the database. **Working of 2PL Protocol** ----- **### Phase 1: Growing Phase** * A transaction acquires locks on the required resources in a specific order. * The transaction can acquire locks on multiple resources, but it must release them before acquiring a new lock. * If a transaction is unable to acquire a lock, it is rolled back. **### Phase 2: Shrinking Phase** * A transaction releases all the locks it has acquired. * The transaction cannot acquire any new locks once it has started releasing locks. **Example** ----- Suppose we have two transactions, T1 and T2, that want to access a shared resource R. In this example, T1 acquires a lock on R and starts executing. T2 requests a lock on R, but it is blocked because T1 has already acquired the lock. **Comparison with Strict 2PL** ----- **Conclusion** ----- The Two-Phase Locking (2PL) protocol is a widely used concurrency control mechanism that ensures the consistency of the database by preventing conflicts between transactions. It consists of a growing phase, where transactions acquire locks, and a shrinking phase, where transactions release locks. While 2PL allows transactions to acquire and release locks in any order, Strict 2PL requires locks to be released in the order they were acquired.

Q16 (10 marks): A customer transfers █20,000 between two bank accounts. Apply the ACID properties to explain how the DBMS guarantees reliable transaction execution.

ACID Properties for Reliable Transaction Execution

===== The ACID properties ensure that database transactions are processed reliably and consistently. These properties are crucial for maintaining data integrity in a multi-user environment. Atomicity ----- * Ensures that a transaction is treated as a single, indivisible unit of work. * If any part of the transaction fails, the entire transaction is rolled back to its previous state. * Example: A customer transfers █20,000 between two bank accounts. If the transfer fails due to insufficient funds, the transaction is rolled back, and the accounts remain unchanged. Consistency ----- * Ensures that the database remains in a consistent state after a transaction is executed. * The transaction must maintain the integrity constraints of the database, such as primary and foreign keys. * Example: A customer's account balance must always be non-negative. Isolation ----- * Ensures that multiple transactions can execute concurrently without interfering with each other. * Each transaction sees a consistent view of the database, as if it were the only transaction executing. * Example: Two transactions can execute concurrently to transfer funds between accounts without affecting each other's results. Durability ----- * Ensures that once a transaction is committed, its effects are permanent and cannot be rolled back. * The database is updated with the changes made by the transaction. * Example: Once the transfer of █20,000 is committed, the accounts are updated, and the transaction is considered complete. Comparison of 2PL and Strict 2PL ----- 2PL (Two-Phase Locking) Protocol * A transaction can acquire locks on data items in two phases: 1. Growing phase : The transaction acquires locks on data items it needs to access. 2. Shrinking phase : The transaction releases locks on data items it has finished accessing. Strict 2PL Protocol * A transaction can acquire locks on data items in two phases: 1. Growing phase : The transaction acquires locks on data items it needs to access. 2. Shrinking phase : The transaction releases locks on data items it has finished accessing, and then acquires locks on data items it needs to modify. Conclusion ----- The ACID properties ensure that database transactions are processed reliably and consistently. The 2PL and Strict 2PL protocols are used to manage concurrency and prevent deadlocks in a multi-user environment. By understanding these concepts, a DBMS can guarantee reliable transaction execution and maintain data integrity.

Q17 (10 marks): Explain the ACID properties of transactions with suitable examples.

Introduction ----- ACID properties are a set of rules that ensure database transactions are processed reliably and consistently. Atomicity ----- * A transaction is treated as a single, indivisible unit of work. * If any part of the transaction fails, the entire transaction is rolled back. * Example: A bank transfer transaction involves debiting one account and crediting another. If the debit fails, the entire transaction is rolled back, and the accounts are restored to their original state. Consistency ----- * A transaction must leave the database in a consistent state. * The database must satisfy all constraints and rules before and after the transaction. * Example: A transaction that updates a customer's address must ensure that the address is valid and consistent with the customer's other information. Isolation ----- * Multiple transactions must be executed independently, without interfering with each other. * Each transaction must see a consistent view of the database, without seeing the effects of other transactions. * Example: Two transactions that update the same account balance must be executed independently, without seeing each other's changes. Durability ----- * Once a transaction is committed, its effects must be permanent and survive even in the event of a system failure. * The database must ensure that all committed transactions are retained and cannot be rolled back. * Example: A transaction that updates a customer's order status must be retained even if the system fails, to ensure that the customer's order is accurately reflected.

Q18 (10 marks): Discuss the atomicity, durability, isolation, and consistency preservation properties of a database transaction.

Transaction Properties ===== A database transaction is a sequence of operations performed on a database. The four main properties of a database transaction are atomicity, durability, isolation, and consistency preservation. Atomicity ----- * A transaction is treated as a single, indivisible unit of work. * If any part of the transaction fails, the entire transaction is rolled back to its previous state. * Atomicity ensures that either all changes are made or none are made. Durability ----- * Once a transaction is committed, its effects are permanent and cannot be rolled back. * Durability ensures that the database remains in a consistent state even after a system failure. Isolation ----- * Each transaction executes independently of other transactions, as if it were the only transaction in the system. * Isolation ensures that the effects of one transaction are not visible to other transactions until it is committed. Consistency Preservation ----- * A transaction must preserve the consistency of the database by maintaining the integrity constraints. * Consistency preservation ensures that the database remains in a consistent state after each transaction. Example ----- Consider a banking transaction where a customer transfers money from one account to another. The transaction involves updating the balances of both accounts. If the transaction fails due to a system failure, the atomicity property ensures that the balances are rolled back to their previous state, maintaining the consistency of the database.

Q19 (10 marks): Apply the Timestamp Ordering protocol to control concurrent transactions with an example.

Timestamp Ordering Protocol ===== The Timestamp Ordering protocol is a concurrency control protocol used in transaction processing systems to ensure that database transactions are executed in a consistent manner. How Timestamp Ordering Protocol Works ----- The Timestamp Ordering protocol assigns a unique timestamp to each transaction before it begins execution. The timestamp is used to determine the order in which transactions can access the database. Key Features of Timestamp Ordering Protocol ----- Example of Timestamp Ordering Protocol ----- Suppose we have two transactions, T1 and T2, that attempt to access the same data item, X. In this example, T1 has a smaller timestamp (10) than T2 (20). Therefore, T1 is executed first, and its read operation on X is allowed. When T2 attempts to write to X, it is blocked until T1 has committed or aborted. Conclusion ----- The Timestamp Ordering protocol is a simple and effective concurrency control protocol that ensures that database transactions are executed in a consistent manner. By assigning a unique timestamp to each transaction and ordering them based on their timestamps, the protocol ensures that conflicts between transactions are resolved in a fair and efficient manner.

Q20 (10 marks): Analyze a schedule where multiple transactions read and write the same data item. Determine whether the schedule is recoverable and explain your reasoning.

Transaction Schedule Analysis ===== A transaction schedule is recoverable if it is serializable, meaning that the schedule can be transformed into a serial schedule without affecting the outcome of the transactions. Properties of a Recoverable Schedule ----- * Consistency : The schedule must preserve the consistency of the database. * Atomicity : The schedule must ensure that either all or none of the

transactions are executed. * Isolation : The schedule must prevent concurrent transactions from interfering with each other. Example Schedule ----- Suppose we have two transactions, T1 and T2, that read and write the same data item, X. Analysis ----- The schedule is not serializable because T1 and T2 interfere with each other's operations. T1 reads X, then T2 writes X, which affects T1's read operation. However, the schedule is atomic and consistent. Conclusion ----- A schedule is recoverable if it is serializable, meaning that it can be transformed into a serial schedule without affecting the outcome of the transactions. However, in this example, the schedule is not serializable due to interference between T1 and T2. Nevertheless, it is atomic and consistent, making it recoverable in a limited sense.

Q21 (10 marks): Explain transaction states additional operations.

Introduction ----- A transaction in a database management system (DBMS) is a sequence of operations performed on the database. It is a logical unit of work that is executed as a single, all-or-nothing unit. Transaction States ----- A transaction can be in one of the following states: * Active : The transaction is currently executing. * Failed : The transaction has encountered an error and cannot continue. * Aborted : The transaction has been terminated prematurely. * Committed : The transaction has completed successfully and its effects are permanent. Additional Operations ----- In addition to the basic transaction states, there are several additional operations that can be performed on a transaction: * Rollback : This operation is used to undo the effects of a transaction. When a transaction is rolled back, all the changes made by the transaction are reversed. * Commit : This operation is used to make the changes made by a transaction permanent. * Savepoint : This operation is used to mark a point in a transaction where the transaction can be rolled back to if necessary. Comparison of Rollback and Commit ----- Conclusion ----- In conclusion, understanding transaction states and additional operations is crucial for managing database transactions effectively. By knowing when to roll back or commit a transaction, developers can ensure that database operations are executed correctly and efficiently.

**Q22 (10 marks): Consider the following banking database schema: •
**ACCOUNT(Account_No, Customer_Name, Balance) •
TRANSACTION(Transaction_ID, Account_No, Amount, Transaction_Type) A
customer transfers █10,000 from Account A101 to Account A102. Write SQL
transaction statements to perform the money transfer using: • START
TRANSACTION / BEGIN • COMMIT • ROLLBACK****

Introduction ----- To transfer money from one account to another, we need to perform a series of database operations, including updating the balance of the sender's account and the receiver's account. We will use a SQL transaction to ensure that either both updates are successful or neither is, maintaining the consistency of the database. SQL Transaction Statements ----- ### Step 1: Start the Transaction ```sql START TRANSACTION; ``` This statement begins a new transaction, which will be committed or rolled back at the end of the transaction. ### Step 2: Update the Sender's Account ```sql UPDATE ACCOUNT SET Balance = Balance - 10000 WHERE Account_No = 'A101'; ``` This statement updates the balance of the sender's account (Account A101) by subtracting █10,000. ### Step 3: Update the Receiver's Account ```sql UPDATE ACCOUNT SET Balance = Balance + 10000 WHERE Account_No = 'A102'; ``` This statement updates the balance of the receiver's account (Account A102) by adding █10,000. ### Step 4: Insert a New Transaction Record ```sql INSERT INTO TRANSACTION (Transaction_ID, Account_No, Amount, Transaction_Type) VALUES ('T001', 'A101', -10000, 'DEBIT'); ``` This statement inserts a new transaction record into the TRANSACTION table, indicating a debit of █10,000 from Account A101. ```sql INSERT INTO TRANSACTION (Transaction_ID, Account_No, Amount, Transaction_Type) VALUES ('T002', 'A102', 10000,

'CREDIT'); `` This statement inserts another new transaction record into the TRANSACTION table, indicating a credit of ■10,000 to Account A102. #### Step 5: Commit the Transaction ``sql COMMIT; `` This statement commits the transaction, making all the changes permanent. #### Alternative: Rollback the Transaction ``sql ROLLBACK; `` This statement rolls back the transaction, undoing all the changes made since the transaction began. Conclusion ----- In this example, we have demonstrated how to use a SQL transaction to transfer money between two accounts. The transaction ensures that either both updates are successful or neither is, maintaining the consistency of the database.

Q23 (10 marks): Explain the ACID properties and concurrency control of database transactions with suitable examples.

ACID Properties ACID stands for Atomicity, Consistency, Isolation, and Durability. These properties ensure that database transactions are processed reliably and securely. Atomicity - A transaction is treated as a single, indivisible unit of work. - If any part of the transaction fails, the entire transaction is rolled back. Consistency - The database remains in a consistent state before and after a transaction. - All transactions must preserve the integrity constraints of the database. Isolation - Each transaction appears to execute in isolation from other transactions. - Other transactions cannot see the effects of a transaction until it is committed. Durability - Once a transaction is committed, its effects are permanent and cannot be rolled back. Concurrency Control Concurrency control ensures that multiple transactions can access the database simultaneously without interfering with each other. Example - Consider two transactions, T1 and T2, that access the same resource, R. - T1 acquires a lock on R and performs an update. - T2 attempts to acquire a lock on R but is blocked until T1 releases the lock. - In Strict 2PL, T1 would have acquired the lock before T2, preventing a deadlock. Conclusion ACID properties ensure reliable and secure database transactions, while concurrency control mechanisms like 2PL and Strict 2PL prevent deadlocks and ensure that transactions execute correctly in a multi-user environment.

Q24 (10 marks): Two transactions T1 and T2 are executed simultaneously on the same database item. • T1 withdraws ■5,000 from an account having balance ■20,000. • T2 deposits ■10,000 into the same account at the same time. Analyze the effect of concurrent execution without proper concurrency control. Explain why concurrency control is essential in DBMS with suitable examples

Introduction ----- Concurrent execution of transactions in a database management system (DBMS) can lead to inconsistent results if not properly controlled. Effect of Concurrent Execution without Concurrency Control ----- When T1 and T2 are executed concurrently without concurrency control, the final balance of the account is inconsistent. T1 withdraws ■5,000, leaving a balance of ■15,000, but T2 deposits ■10,000, resulting in a balance of ■30,000. Why Concurrency Control is Essential ----- Example of Concurrency Control ----- To prevent inconsistent results, concurrency control mechanisms such as locking or timestamping can be used. For example, a lock can be acquired on the account before T1 and T2 are executed, ensuring that only one transaction can access the account at a time. Conclusion ----- Concurrency control is essential in DBMS to ensure consistent results and maintain data integrity. Without concurrency control, concurrent execution of transactions can lead to inconsistent results, data inconsistency, and loss of data integrity.

Q25 (10 marks): What is NoSQL? Classify and explain the various types of NoSQL databases.

NoSQL Databases ===== NoSQL databases are a type of database that does not follow the traditional structured query language (SQL) used in relational databases. They are designed to handle large amounts of unstructured or semi-structured data. Types of NoSQL Databases -----
Key-Value Stores * Store data as a collection of key-value pairs * Simple and fast data retrieval * Examples: * Riak * Amazon DynamoDB ####
Document-Oriented Databases * Store data in self-describing documents * Flexible schema * Examples: * MongoDB * Couchbase ####
Column-Family Databases * Store data in columns instead of rows * Optimized for large amounts of data * Examples: * Cassandra * HBase ####
Graph Databases * Store data as nodes and edges * Optimized for complex relationships * Examples: * Neo4j * Amazon Neptune ####
Multi-Model Databases * Support multiple data models * Flexible schema * Examples: * OrientDB * Cosmos DB
Comparison of NoSQL Databases
----- Conclusion NoSQL databases offer a variety of data models and flexible schema, making them suitable for handling large amounts of unstructured or semi-structured data. They are designed to scale horizontally and provide high performance.

Q26 (10 marks): A distributed banking application experiences network partition failures. a) Identify the CAP properties preferred by the application. b) Explain why these properties are suitable for banking transactions.

Distributed Banking Application CAP Properties

===== a) CAP Properties Preferred by the Application ----- The CAP properties preferred by the distributed banking application are: * Availability (A) : The application must ensure that banking transactions are available to users at all times, even in the presence of network partition failures. * Consistency (C) : The application must maintain data consistency across all nodes in the distributed system to prevent incorrect transactions. * Partition Tolerance (P) : The application must tolerate network partition failures and ensure that transactions are not rolled back due to network failures. b) Suitability of CAP Properties for Banking Transactions ----- The CAP properties are suitable for banking transactions because: * Availability ensures that users can access their accounts and perform transactions at all times. * Consistency ensures that data is accurate and up-to-date, preventing incorrect transactions. * Partition Tolerance ensures that transactions are not rolled back due to network failures, maintaining the integrity of the system. In conclusion, the distributed banking application requires high availability, consistency, and partition tolerance to ensure that transactions are processed correctly and efficiently, even in the presence of network partition failures.

Q27 (10 marks): Explain the CRUD operations supported by MongoDB. Illustrate each operation with suitable examples.

Introduction ----- MongoDB is a NoSQL database that supports various operations for managing data. The primary operations supported by MongoDB are Create, Read, Update, and Delete (CRUD). CRUD Operations -----
Create (Insert) Operation * The Create operation is used to insert new documents into a collection. * It allows users to add new data to the database. * Example: * Insert a new document into the `students` collection: ``bash
db.students.insertOne({ name: "John Doe", age: 20, city: "New York" }) `` * Result: A new document is inserted into the `students` collection. ####
Read (Find) Operation * The Read operation is used to retrieve existing documents from a collection. * It allows users to fetch data from the database. * Example: * Find all documents in the `students` collection: ``bash
db.students.find() `` * Result: All documents in the `students` collection are retrieved. ####
Update

Operation * The Update operation is used to modify existing documents in a collection. * It allows users to update data in the database. * Example: * Update a document in the `students` collection: ````bash db.students.updateOne({ name: "John Doe" }, { $set: { age: 21 } }) ```` * Result: The document with the name "John Doe" is updated with a new age. #### Delete Operation * The Delete operation is used to remove existing documents from a collection. * It allows users to delete data from the database. * Example: * Delete a document from the `students` collection: ````bash db.students.deleteOne({ name: "John Doe" }) ```` * Result: The document with the name "John Doe" is deleted from the `students` collection. Conclusion ----- In conclusion, MongoDB supports four primary operations: Create, Read, Update, and Delete (CRUD). These operations enable users to manage data in the database, including inserting new documents, retrieving existing documents, modifying existing documents, and deleting existing documents.

Q28 (10 marks): An online banking system must maintain accurate account balances despite network partitions. Apply the CAP theorem and identify the most suitable CAP properties for the system.

CAP Theorem and Online Banking System

===== The CAP theorem states that it is impossible for a distributed database system to simultaneously guarantee all three of the following: * Consistency : Every read operation will see a consistent view of the system. * Availability : Every request receives a response, without the guarantee that it contains the most recent version of the information. * Partition Tolerance : The system continues to function and make progress even when there are network partitions. Applying CAP Theorem to Online Banking System ----- To maintain accurate account balances despite network partitions, the online banking system must prioritize consistency and availability. * Consistency : The system must ensure that every read operation sees a consistent view of the system, which means that the account balances must be up-to-date and accurate. * Availability : The system must ensure that every request receives a response, without the guarantee that it contains the most recent version of the information. This means that the system must be able to handle a large number of concurrent transactions and provide a response to each request. * Partition Tolerance : The system must continue to function and make progress even when there are network partitions. This means that the system must be able to handle network failures and continue to operate. Most Suitable CAP Properties for the System

----- Based on the requirements of the online banking system, the most suitable CAP properties are: * Consistency : High * Availability : High * Partition Tolerance : High This means that the system must prioritize consistency and availability, and be able to handle network partitions. Conclusion ----- In conclusion, the CAP theorem provides a framework for understanding the trade-offs between consistency, availability, and partition tolerance in distributed database systems. For an online banking system, the most suitable CAP properties are consistency, availability, and partition tolerance. This means that the system must prioritize consistency and availability, and be able to handle network partitions.

Q29 (10 marks): A banking application requires that account balances must always remain accurate, even during server communication failures. However, during such failures, some users may not be able to access the system temporarily. Apply the CAP Theorem to: Identify the CAP properties preferred by the banking system. Explain why this choice is suitable for banking applications. Name a suitable database system for this scenario.

CAP Theorem Application in Banking System The banking system requires high availability, consistency, and partition tolerance. However, these properties are conflicting, and a trade-off is necessary. CAP Properties Preferred by the Banking System The banking system prefers high

consistency to ensure that account balances remain accurate. High availability is also essential to minimize downtime during server communication failures. Partition tolerance is necessary to ensure that the system remains functional even during network partitions. Why This Choice is Suitable for Banking Applications High consistency ensures that account balances are accurate, even during concurrent updates. High availability minimizes downtime, allowing users to access the system despite server communication failures. High partition tolerance ensures that the system remains functional even during network partitions, preventing data loss. Suitable Database System A suitable database system for this scenario is Google's Spanner . Spanner is a globally distributed relational database service designed for large-scale applications. It provides high consistency, availability, and partition tolerance, making it suitable for banking applications. Conclusion In conclusion, the banking system requires high consistency, availability, and partition tolerance. Google's Spanner is a suitable database system that meets these requirements, ensuring accurate account balances and minimizing downtime during server communication failures.

Q30 (10 marks): Consider a MongoDB collection named books with the following document structure: { "_id": 1, "title": "DBMS", "author": "Navathe", "price": 550 } Write MongoDB queries to perform the following CRUD operations: 1. Insert a new book document into the books collection. 2. Display all documents from the collection. 3. Update the price of the book "DBMS" to 600.

Introduction Transaction processing systems are designed to handle large databases and hundreds of concurrent users. MongoDB is a popular NoSQL database that supports transactional capabilities. In this section, we will discuss the basics of transaction processing and perform CRUD operations on a MongoDB collection. ### Transaction Properties A transaction is a logical unit of database processing that consists of one or more database operations. The key properties of a transaction are: * Atomicity : A transaction is treated as a single, indivisible unit of work. * Consistency : A transaction must leave the database in a consistent state. * Isolation : Multiple transactions must be executed independently, without interference from other transactions. * Durability : Once a transaction is committed, its effects must be permanent. ### Creating a MongoDB Collection To perform CRUD operations on a MongoDB collection, we first need to create the collection. Let's create a collection named "books" with the following document structure: ``json { "\_id": 1, "title": "DBMS", "author": "Navathe", "price": 550 } `` ### CRUD Operations ##### Insert a New Book Document To insert a new book document into the "books" collection, we can use the following MongoDB query: ``bash db.books.insertOne({ "\_id": 2, "title": "Database Systems", "author": "Silberschatz", "price": 650 }) `` ##### Display All Documents from the Collection To display all documents from the "books" collection, we can use the following MongoDB query: ``bash db.books.find() `` This will return the following output: ``json [ { "\_id" : 1, "title" : "DBMS", "author" : "Navathe", "price" : 550 }, { "\_id" : 2, "title" : "Database Systems", "author" : "Silberschatz", "price" : 650 } ] `` ##### Update the Price of the Book "DBMS" To update the price of the book "DBMS" to 600, we can use the following MongoDB query: ``bash db.books.updateOne({ "title": "DBMS" }, { \$set: { "price": 600 } }) `` ### Conclusion In this section, we discussed the basics of transaction processing and performed CRUD operations on a MongoDB collection. We created a collection, inserted a new document, displayed all documents, and updated the price of a book. These operations demonstrate the capabilities of MongoDB as a transactional database.

Q31 (10 marks): Explain the characteristics of NoSQL systems with suitable examples. Bring the difference between SQL and NoSQL.

NoSQL Systems NoSQL systems are designed to handle large amounts of unstructured or semi-structured data and provide high scalability and flexibility. Characteristics of NoSQL Systems * Schema-less : NoSQL databases do not require a predefined schema, allowing for flexible data

modeling. * Distributed : NoSQL databases are designed to scale horizontally, allowing for easy addition of new nodes to handle increased traffic. * High Performance : NoSQL databases are optimized for high-performance and low-latency data access. * Flexible Data Model : NoSQL databases support a variety of data models, including key-value, document, graph, and column-family stores. Examples of NoSQL Systems * MongoDB : A popular NoSQL database that uses a document-oriented data model. * Cassandra : A distributed NoSQL database that uses a column-family data model. * Redis : An in-memory NoSQL database that uses a key-value data model. Comparison of SQL and NoSQL Systems Conclusion NoSQL systems are designed to handle large amounts of unstructured or semi-structured data and provide high scalability and flexibility. They are optimized for high-performance and low-latency data access and support a variety of data models.

Q32 (10 marks): Illustrate the working of a NoSQL graph database and explain Neo4j with a suitable example.

NoSQL Graph Database ===== A NoSQL graph database is a type of database that stores data as a graph, where nodes represent entities and edges represent relationships between them. Working of a NoSQL Graph Database ----- * Key Features: * Schema-less : No fixed schema, allowing for flexible data modeling. * High-performance : Optimized for querying and traversing large graphs. * Scalability : Designed to handle large amounts of data and scale horizontally. * Data Model: * Nodes : Represent entities, such as users, products, or locations. * Edges : Represent relationships between nodes, such as friendships, purchases, or connections. * Querying: * Cypher Query Language : Used to query and traverse the graph. * Traversal : Allows for efficient navigation of the graph. Neo4j ----- Neo4j is a popular NoSQL graph database that stores data as a graph. Key Features of Neo4j ----- Example: Social Network ----- Suppose we have a social network where users can follow each other. We can model this as a graph with users as nodes and follow relationships as edges. * Nodes: * User 1 (node: "Alice") * User 2 (node: "Bob") * User 3 (node: "Charlie") * Edges: * Alice -> Bob (edge: "follows") * Bob -> Charlie (edge: "follows") We can query the graph to find all users that Alice follows, or all users that follow her. This is just a simple example, but graph databases like Neo4j can handle much more complex queries and relationships. Conclusion ----- NoSQL graph databases like Neo4j provide a powerful way to store and query complex data relationships. With its flexible schema, high-performance querying, and scalability, Neo4j is a popular choice for many applications, including social networks, recommendation engines, and more.

Q33 (10 marks): Consider a MongoDB collection named restaurants with the following document structure: { "_id": 101, "name": "Spice Garden", "city": "Bangalore", "rating": 4.2 } Write MongoDB queries to perform the following operations: 1. Insert two restaurant documents into the collection. 2. Retrieve the details of restaurants located in "Bangalore". 3. Delete all restaurants having rating less than 3.5.

Introduction ----- Transaction processing in MongoDB provides a mechanism for describing logical units of database processing. It ensures that database operations are executed as a single, all-or-nothing unit of work. Transaction Properties ----- ### Atomicity * Ensures that database operations are executed as a single, all-or-nothing unit of work. * If any operation fails, the entire transaction is rolled back. ### Consistency * Ensures that the database remains in a consistent state after the transaction is completed. * Ensures that the transaction does not violate any database constraints. ### Isolation * Ensures that multiple transactions can execute concurrently without interfering with each other. * Ensures that each transaction sees a consistent view of the database. ### Durability * Ensures that once a transaction is committed, its effects are

permanent and cannot be rolled back. Creating a Transaction in MongoDB

```
----- ``javascript db.restaurant.startSession(); var session =  
db.restaurant.getStartedSession(); try { session.startTransaction(); db.restaurant.insertOne({ "_id":  
102, "name": "Spice Bites", "city": "Bangalore", "rating": 4.5 }); db.restaurant.insertOne({ "_id": 103,  
"name": "Spice Delights", "city": "Mumbai", "rating": 4.8 }); session.commitTransaction(); } catch (e)  
{ session.abortTransaction(); } session.endSession(); `` Retrieving Details of Restaurants in  
"Bangalore" ----- ``javascript db.restaurant.find({ city:  
"Bangalore" }); `` Deleting Restaurants with Rating Less Than 3.5
```

```
----- ``javascript db.restaurant.deleteMany({ rating: { $lt: 3.5 } });
```

`` Comparison of 2PL and Strict 2PL ----- In 2PL, locks are acquired and released in a specific order, which can lead to deadlocks and starvation. In Strict 2PL, locks are acquired and released in a specific order, and the system ensures that a transaction never waits indefinitely for a lock to be released. Conclusion ----- Transaction processing in MongoDB provides a mechanism for describing logical units of database processing. It ensures that database operations are executed as a single, all-or-nothing unit of work. The properties of atomicity, consistency, isolation, and durability ensure that the database remains in a consistent state after the transaction is completed. The comparison of 2PL and Strict 2PL highlights the importance of ensuring that locks are acquired and released in a specific order to prevent deadlocks and starvation.